

kayfabe — the v2 architecture, the machine as it runs, and the surface we present

A working document, assembled 2026-09-20 (w818). Written to be argued with.

branch w749-fable-legb

Contents

Read this first	2
The markers, used consistently throughout	2
What is knowingly missing	2
Where the code actually stands, by the numbers	2
Part 1 — The v2 architecture (the proposal)	4
0. The one-paragraph version	4
1. What we are allowed to be wrong about, and what we are not	4
2. Memory: one store, identity offsets	4
3. Addresses: the host page tables ARE the model	5
4. The three channel kinds	5
5. The scrub: the guest already does it — we must stop lying about it	6
6. The hot path, end to end	6
7. What remains unresolved — named, not designed away	7
8. What v2 keeps, deliberately	7
Part 2 — The machine, as it is	8
0. How to read this	8
2. Processes and threads	10
3. The vCPU trap path, step by step	11
4. Locks, and the blocking rules	13
5. Data structures	14
6. Installing a mapping, and observing a completion	16
7. The proposal, and the arguments against it	18
8. Open problems — named, not designed away	19
Part 3 — The surface we present	20
1. GSP RPCs	20
2. RM control commands	21
3. MMIO registers	22
4. Channel / pushbuffer operations	23
5. □ Where this inventory is knowingly incomplete	24
Part 4 — The scrub question (open owner ruling)	25
What is measured	25
Why this is a decision and not a bug	25
The argument each way	25
If the ruling is “\$46 wins”, the work and its gate	26
What must not be read into this page	26

Read this first

This bundle is four documents, in the order they should be read:

1. **The v2 architecture** — the design I propose. Short, and the part to attack first.
2. **The machine, as it is** — what the code actually does today, from a file:line survey. Every place the code contradicts the proposal is marked.
3. **The surface we present** — every GSP RPC, RM control command, MMIO register and push-buffer method we implement: what it is, what we do with it, and the plan for it.
4. **The scrub question** — one open owner ruling, isolated because it is a live security defect and it is the smallest change on the list.

The markers, used consistently throughout

marker	meaning
[MEASURED]	a number from a real boot, with the run that produced it
[CODE]	read out of the tree, with file:line
[PROPOSE]	a design opinion. These are the arguable ones.
[UNVERIFIED]	the survey could not establish it. Deliberately not filled in with a guess.
∅	a refusal, a deletion, or a thing that is <i>not</i> true
□	a load-bearing fact
□	a hazard, or a place a future edit would break something

What is knowingly missing

∅ Stated up front, because a document that hides its gaps cannot be argued with:

- **Registers are GA106 only.** Three other chip modules exist and were not enumerated.
- **The 160-row capability allowlist is not transcribed.** It gates *admission*, not service.
- **A second PushMethod consumer** (kayfabe-rt/src/ceutils.rs) was confirmed to use the same vocabulary but **not** audited for additional method addresses.
- **Three live contradictions are left in, unresolved**, because only one side of each can be true and I could not determine which. They are flagged where they appear; the most consequential is whether guest MMIO traps run with the QEMU BQL held.
- **No performance model.** The one number here is a 205.7 μs page-table walk and a 1.62 s worst-case trap; there is no throughput or syscall-cost measurement for the proposed design.

Where the code actually stands, by the numbers

[MEASURED 2026-09-20]

	lines
crates/*/src — code	144,175
crates/*/src — comments	125,503 (45 % of the file bytes)
tests (crates/*/tests + tests/)	182,052
total Rust	463,996

⇒ A 50k-line target is a **~3x reduction of actual code**, not 9x — the comment density is doing a lot of the apparent bulk, and the comments are the part worth keeping. The deletions proposed here (joins, the address table, VA translation, the CPU executor, the isolate IPC) are concentrated in

the four largest crates: `kayfabe-isolate-host` (46,895), `kayfabe-abi` (43,588), `kayfabe-qemu-raw` (37,057) and `kayfabe-device` (27,496).

Part 1 — The v2 architecture (the proposal)

STATUS: LIVE, written 2026-09-20 (w815). Supersedes ARCHITECTURE_from_scratch.md and l1_architecture_summary.md, both of which describe the two-worlds framebuffer and the address table this document deletes.

∅ This is a DESIGN document, not a description of the tree as it stands. Where the code disagrees with this document, the code is what is scheduled to change — each such place is marked **[ROT]** with what it costs today. Nothing here is a claim that it already works.

0. The one-paragraph version

A stock NVIDIA driver runs in a KVM guest against an emulated GPU and a faked GSP. Guest video memory **is** one reserved device-local RM object on the host, and a guest framebuffer offset **is** a file offset into it — the identity is the whole memory model. The guest's own page tables are walked on the GPU (205.7 μ s) and the diff is applied as mmap/munmap against a host address space, so **the host page tables are the address model** and nothing mirrors them. A doorbell is never delayed for that refresh. The only ordering that is load-bearing anywhere is that a physical page may not reach a second guest client until its old translations are gone and it has been **scrubbed on hardware**.

1. What we are allowed to be wrong about, and what we are not

□□□□ **THE CENTRAL RULE OF v2, and it is the guest's own rule.** ogkm — the honest guest driver — already accepts that until an invalidate has *completed*, addresses are **undefined**. It does not accept that they may be **unsafe**. Concretely, a translation may resolve to the old page or the new page; it may never resolve to a page neither mapping ever referenced.

	permitted	forbidden
a stale translation (old page)	□ the guest tolerates it by construction	
a fresh translation (new page)	□	
a fault	□ — it is not a third thing	
a page neither mapping referenced		∅∅∅ this is the only memory-safety failure

⇒ Everything expensive in v1 bought *freshness*, which the guest never asked for. v2 spends nothing on freshness.

□ And the third row is not something we must build machinery to prevent: the guest already prevents it, by scrubbing a page before handing it on. We reach it only by **telling the guest a scrub happened when it did not** (§5). The fix is to stop lying, not to add an invariant.

∅ **[measured w814] We are currently violating it.** --cross-client-leak: client A writes through its own VA and client B's object changes (R5b B saw A's 2nd). The same arm on bare metal reports R5b ISOLATED ... leak A<-B=false leak B<-A=false. That is the third row, and it is ours.

2. Memory: one store, identity offsets

- Guest vidmem is **one** reserved device-local RM object, allocated once at realize.
- Guest FB offset **is** the offset into that object. There is no allocator, no per-page fabrication, no second world.

- Guest system is the hypervisor's memfd, reached by a **layout** (GPA range $\rightarrow$ fd + offset), which is O(memslots) and given to us, not observed.

What this deletes outright:

- $\emptyset$ **Joins.** join_one_fb_leaf, FbJoinArm, join_release and its supersede/alias arms, the ALREADY JOINED / CANDIDATE accounting. A join exists to give an *emulated framebuffer page* a host object; under one store every page already has one, by construction. **[ROT]** [measured w812] three gates of that machinery were what kept the guest's CE work off the GPU: join_one_fb_leaf returned None for every leaf, operands stayed /FakeFramebuffer, every copy graded CeExecutor::Ours.
- $\emptyset$ **The two-worlds framebuffer** (twoworlds.rs, the arena arm, demand-fill mirrors).
- $\emptyset$ **FbStoreArm::Arena** — the surviving control arm of a switch that has been made.

3. Addresses: the host page tables ARE the model

$\square$ **There is no address table.** The PTX walk kernel walks the guest's page tables on the GPU and produces a **diff**. The diff is a set of mmap/munmap calls. After applying it, the answer to “*what does this VA resolve to*” lives in exactly one place: the host address space.

Every field a table would have held is already present at the moment you act:

a table would store	where v2 gets it
target of the mapping	identity — mmap(store, offset = fb_offset); no lookup
system target	the static layout; O(memslots), not O(pages)
permissions, aperture, size	the PTE just walked
“it went away”	munmap

What this deletes outright:

- $\emptyset$ **VA $\rightarrow$ GPA translation on the submission path.** Channels are born in the birth client with the guest's own VA space duped (route K), so the guest's VA is the host's VA. A translation step would be an identity function that can fail.
- $\emptyset$ **The operand gate.** We currently pre-validate CE operands — “*the copy named an operand page this channel's VA space binds nowhere*” — because we do not trust the mapping to exist. In v2 the GPU resolves operands itself and **faults** if we were wrong, which is what real hardware does. $\square$ The C artifact went green *without servicing a single GPU fault*, because a wholesale sweep made faults impossible as a class rather than by checking each one.
- $\emptyset$ **The publication machinery:** publication epochs, VasPublishArm, the w318 dirty gate, pt_sweep_skip, candidates/published/refused census rows, UnknownPdb, CrossesEnd. **[ROT]** these are *reconciliation* failures — two copies of one fact disagreeing — and a design with nothing to reconcile cannot have them. Most of 2026-09-19/20 was spent inside them.
- $\emptyset$ **Pdb as an identity.** **[ROT]** Pdb currently means both “*which address space*” and “*the physical address of its page directory*”, and vas.pdb.unwrap_or(Pdb(0)) at eight call sites makes Pdb(0) mean both “*declared at framebuffer offset 0*” and “*no declaration yet*”. [measured w812] that single collision produced **four** consecutive wrong diagnoses, each a true statement about the wrong population. In v2 an address space is named by its guest resource key; a root is Option<Pdb> and is needed **only** to seed a walk.

4. The three channel kinds

kind	ring	who executes	we inspect
Passthrough	the guest's own	the GPU	never — parsing, not placement, is what is forbidden
Translated	ours; the guest's is a fake in GPGA	the GPU, from a rewritten pushbuffer	every entry
Emulated	ours	us — it is a function we implement	n/a: there is no GPU counterpart

□ Emulated is for channels that have no hardware meaning for us to forward — the UVM kernel channel, things we must stub. ∅ It is **not** a fallback for “we could not translate this”. A translated entry we cannot rewrite is refused by name, or forwarded to a channel that deliberately faults so the guest's own error reporting works untouched.

[ROT] CeExecutor::Ours — the CPU byte-copy — is not one of these three kinds. Under §46 real kernel-channel work is never executed on the CPU. When that lands, CeExecutor::Ours, CE-LOCAL-REFRESH (132 per run), DEFERRED-LOCAL, the CE-split/commit_ce path and KAYFABE_CE_EXECUTOR=local all delete together.

5. The scrub: the guest already does it — we must stop lying about it

□□□□ **We own no scrub policy.** The guest's own RM scrubs a page before it reaches another client, and it orders that correctly by itself. There is no invariant for us to enforce, no “tear down translations then scrub then re-issue” sequence for us to implement, and no lifetime question for us to answer.

∅ **Our only failure is that we forge the completion.** [measured w813] the kernel tokens carrying that work are tok=0x00010001 emulated=74 forwarded=0 and tok=0x00010004 emulated=66 forwarded=0 — 140 doorbells rung and **not one sent to the GPU**. We report the scrub complete; no bytes were cleared. The guest then reuses the page, correctly by its own rules, and it still holds the previous client's data.

⇒ **That is the cross-client leak.** R5b B saw A's 2nd is not a staleness bug and not an ordering bug — it is a page handed over unscrubbed because we said it had been scrubbed.

The fix is one thing: let the scrub's work reach the hardware. FwdFault::SystemDataPlane refuses the system proc a data plane, implementing ll1_concurrency.md §12.26 — “*the CeUtils scrub ... is forged ... never forwarded, so the system proc never mints host memory.*” §46 names that exact example and reverses it: “*Ceutils scrub is not a no-op, it genuinely needs to clear memory. Same for kernel ce.*” ∅ §12.26's stated reason is obsolete under one store anyway: the store is minted **once at realize**, so a pin mints nothing.

□ **The gate already exists.** Those two tokens must read forwarded > 0. Nothing new needs building to know whether this is fixed.

∅ And the unmap half needs no special mechanism either. **[ROT]** the_store_slice_unmap_was_never_wired records that nothing ever removes a store slice — but in v2 removal is simply the munmap half of the walk diff (§3). It stops being its own mechanism with its own bugs.

□ Out of scope, and must not be conflated with it: ce_copy(Ours) refusing inside the isolate is a **different** refusal — the sandbox declining to hold guest memory — and stays.

6. The hot path, end to end

1. Guest writes a doorbell (NV_VIRTUAL_FUNCTION_DOORBELL, BAR0 0x00BB0090).

2. The trap does one allowlisted thing — update a queue, wake a worker, one dword for a passthrough doorbell, or a synchronous read-register write (§41). **No blocking work ever runs on a vCPU.**
3. A worker picks it up. **It does not wait for a page-table refresh** (§1).
4. Passthrough: the GPU reads the guest's ring. Translated: we read the ring, rewrite operands, submit. Emulated: we perform the function.
5. Completion is the guest's own semaphore, at the guest's own address. We do not forge it and we do not forward the guest's spin.

□ Refresh runs **beside** this, not inside it: walk, diff, apply, repeat. A doorbell that races a refresh gets old-or-new, which §1 permits.

7. What remains unresolved — named, not designed away

1. **Doorbell → channel.** A token must still reach a channel object. That is a small fixed-size map, not an address table, but it has an owner and a lifetime and this document does not specify them.
 2. **The PTX diff's previous state.** A diff needs something to diff *against*. That is one shadow of the last walk — O(mapped pages) — and it is the one structure v2 keeps. □ It must never be consulted to *answer* a translation; the moment anything reads it on the submission path it has become the address table again.
 3. **Instance-block / PDB discovery.** MAP_MEMORY_DMA is a HAL stub on GSP-client parts, so a client-allocated VA space is never declared to us — [measured w811] zero SET_PAGE_DIRECTORY events in a whole boot. The walk needs a root. Unsolved.
 4. **Syscall cost of the diff.** N changed pages ⇒ N mappings unless runs are coalesced. ∅ **Unmeasured.** The walk is 205.7 μs and page tables change far less often than doorbells ring, so the diff should be small and bursty — but that is a prediction, not a measurement.
 5. **Multi-GPU and non-GA10x parts.** The chip table holds exactly one profile (GA106); ad10x, gh100, gb20x exist in kayfabe-chips and are not in it.
-

8. What v2 keeps, deliberately

∅ The instruments, and they are the reason this rewrite is safe to attempt at all: the doorbell ledger (forwarded= per token), the censuses, refusals **by name**, the bare-metal baseline, the hardware gate, the 30-arm suite. [measured 2026-09-20] the bare-metal baseline is what turned nine guest failures from opinion into adjudication.

□ Also kept: every **measured** fact in a comment. The mechanisms in this tree are the liability; the measurements are the asset, and a rewrite that discards them pays for them twice.

Part 2 — The machine, as it is

STATUS: LIVE, 2026-09-20 (w816). Companion to THE_ARCHITECTURE_v2.md (the proposal) and THE_SURFACE_v2.md (the per-item inventory). **This** document is the one an engineer should attack: it describes the system as it actually runs, from a file:line survey, and marks every place I propose to change it.

0. How to read this

Three markers, used consistently:

marker	meaning
[MEASURED]	a number from a real boot, with the run that produced it
[CODE]	a fact read out of the tree at w749-fable-legb, with file:line
[PROPOSE]	my design opinion. Argue with these.
□ [UNVERIFIED]	the survey could not establish it. Not filled in with a guess.

∅ Two contradictions found while writing this are left **in**, unresolved, because only one side of each can be true and I could not determine which:

1. **The BQL.** shim.rs and docs/design/blocking_and_completion_model.md:16 both state “*every guest MMIO write arrives with the QEMU BQL held.*” But qemu/hw/misc/nvkvm/nvkvm.c:1118 calls memory_region_enable_lockless_io on **every** region, the realize-time self-check *refuses the device* if the flag is missing (nvkvm.c:1568), and nvkvm.c:392 states as measured fact: “*these callbacks run CONCURRENTLY on every vCPU with NO BQL.*” □ If the lockless path is live, every “the whole VM is frozen during a trap” argument in this tree is stale, and the shared-lock findings in §4 get much sharper.
2. **Vas is keyed by (GpuId, ResourceKey)** [CODE gpu.rs:1761], but at least three doc comments still say (GpuId, Pdb) (gpu.rs:1760, :250, :275). □ I did not determine whether any reachability argument still depends on the stale key.

□ And a third, which I fixed while writing: ScratchpadArm::Off is documented “*The default*” while scratchpad_from(None) returns Measure. That stale line made one of my own surveys report that the single store is off by default — i.e. that the central memory design does not run. It is the **third** stale-default of the day, after IsolatePlane::Stillborn and KAYFABE_VAS_OWNER=isolate. ## 1. Vocabulary — what the nouns actually are

∅ Written for a reader who does not know NVIDIA’s constant names. Every term below appears in the rest of the document and in the code; this is the only place they are defined.

GPU, host, guest. The *host* is the machine with the real card. The *guest* is a VM running a **stock, unmodified** NVIDIA driver. The guest believes it owns a GPU. It does not — it owns our emulation of one, and real work is carried out on the host’s card by unprivileged host processes we control.

RM (Resource Manager). The core of NVIDIA’s driver. Everything the driver does is “allocate an object of class X under parent Y”, “call control command Z on object W”, “free it”. Objects form a tree rooted at a *client*. This object protocol is the control plane, and it is what we emulate.

Client / handle / object. A *client* is a driver-level session (roughly: one open of /dev/nvidiactl). A *handle* is a 32-bit number the guest picks to name an object **within** its client. Handles are per-client, reusable after free, and therefore not identities — two different objects can wear the same number at different times. This is why the code keeps “incarnations” rather than trusting handle values.

GSP (GPU System Processor). On modern cards, a RISC-V core **on the GPU** runs a firmware that does most of RM's privileged work. The host driver stops touching hardware directly and instead sends the GSP messages over a queue in memory. □ **We impersonate the GSP.** That is the central trick: we do not emulate a GPU register-by-register, we answer the driver's *messages* the way its firmware would.

RPC. One of those messages. Each has a numeric function id (e.g. 0x67 = "allocate an object"). The guest writes a message into a ring buffer in its own memory and pokes a register; we read it, act, and write a reply back into the ring.

Control command. A sub-kind of RPC: "run command 0x20800102 against object H, here are the parameters." These are the bulk of the driver's vocabulary — hundreds of them, each with its own parameter struct. Most are *queries* ("how much framebuffer is there?", "what are this copy engine's capabilities?").

BAR0 / BAR1 / BAR2. PCI memory windows the card exposes to the CPU. - **BAR0** is the register aperture — 16 MiB of control registers. A guest read or write here traps to us. - **BAR1** is a window onto video memory, so the CPU can read/write VRAM. - **BAR2** is a second window, used for *instance memory* (the structures describing channels).

Framebuffer / VRAM / vidmem. The card's own memory. A *framebuffer offset* is a byte offset into it. □ In our design guest vidmem is **one** big host allocation, and a guest framebuffer offset is simply an offset into that allocation — the identity is the entire memory model.

Sysmem. Ordinary system RAM the GPU can also read, via an IOMMU-ish mapping. For us this is the guest's RAM, which the hypervisor hands us as a shareable file descriptor (memfd).

GPU virtual address (VA) and the page tables. The GPU has its own MMU. Work submitted to it uses *virtual* addresses, translated by page tables the driver builds in memory. A **PDB** (page directory base) is the physical address of the root of one such tree — i.e. "which address space". The GPU's TLB caches translations, so after changing tables the driver issues a **TLB invalidate**.

VA space. One page-table tree = one address space. A guest process typically has one; the kernel driver has others. Two channels sharing a VA space see identical translations.

Channel / GPFIFO. A *channel* is a submission queue to an engine. The driver writes command packets into a **pushbuffer** in memory, then appends an entry to a ring called the **GPFIFO** pointing at that pushbuffer. Hardware consumes the ring.

USERD. A small per-channel structure holding the ring cursors: GP_PUT (how far the driver has filled) and GP_GET (how far hardware has consumed). The driver writes GP_PUT; hardware writes GP_GET.

Doorbell. After updating GP_PUT, the driver writes one 32-bit word to a register to say "there is new work on channel N". That write is a *doorbell*. It is the hottest trap in the system — and on a real card it is a posted write that costs the driver almost nothing.

Engines: CE and GR. - **CE (Copy Engine)** — a DMA engine. It moves bytes between addresses. LAUNCH_DMA is its "go" command; operands (source, destination, length) are loaded into it by preceding commands. - **GR (Graphics/Compute)** — runs shaders/kernels. CUDA work lands here.

Semaphore release. How software learns an engine finished. The driver puts a "when done, write value V to address A" command in the pushbuffer, then **spins** reading address A until it sees V. ∅ This matters enormously for us: the wait is the *guest's own* spin on *its own* memory, so if the value appears the guest proceeds, and if it never appears the guest hangs. We must never forge V for work that genuinely had to happen.

Scrub. Before RM hands a page of video memory to a different client, it clears it — with a copy-engine fill, on a kernel-owned channel. This is what stops one process reading another's old data. □ The guest does this itself and orders it correctly; our only job is not to lie about whether it happened.

WPR2 (Write-Protected Region 2). A region of VRAM the GSP firmware is loaded into, fenced off by hardware. The driver checks registers to see whether it is already up. It only resets on a full

device reset, which is why each clean run needs a fresh VM boot.

CeUtils. RM’s internal helper that owns the kernel-side copy-engine channels — the ones used for scrubbing and other housekeeping. Its channels belong to no guest process.

UVM (Unified Virtual Memory). A second guest driver module (`nvidia-vm`) providing managed memory. It has its own `ioctl` surface and its own channels, and CUDA uses it heavily.

Xid. An NVIDIA error code reported by the driver when hardware faults. `Xid 31` is a GPU memory-access fault — the engine touched an address the page tables did not map. $\emptyset$ For us an `Xid` on the host is a signal we got a mapping wrong.

2. Processes and threads

2.1 The processes

A full Mode-2 boot is **one QEMU process** with our Rust archive statically linked in as a QOM device, plus **N+1 sandboxed child processes**.

process	count / lifetime	opens <code>/dev/nvidia*?</code>	what it is for
VMM (QEMU)	1 per VM	No [CODE]	owns the guest, KVM fd, all guest RAM, the MMIO traps, the pure core, the doorbell table, the BAR mirrors, and the isolate factory holds the ONE reserved video-memory object, the walk kernel and (when armed) a CUDA context
Scratchpad isolate	1 per (VM, GPU), spawned at PCI realize	Yes	that guest process’s RM client, its channels, its VA spaces
Per-proc isolate	1 per (guest process, GPU), lazy	Yes	

[CODE] Every `/dev/nvidia*` open in the workspace is in `kayfabe-isolate-host` (the child side) or the standalone ladder tool. **The VMM issues no RM `ioctl`.** It holds descriptors the children hand up — sealed `memfds` and armed device nodes — which it only `mmaps`, never `ioctl`s (`export.rs:31-42`).

□ **The “birth client B” is not a process.** It is an RM client handle plus two character-device descriptors, minted **inside a per-proc isolate** (because RM stamps `pClient->ProcID` from the creating task) and passed to the scratchpad by `SCM_RIGHTS` [CODE `child.rs:722-740`].

[PROPOSE] Isolates stay. Their founding reason is **VA identity** — NVIDIA, and UVM especially, treats host VA and guest VA as one address, so each guest process needs its own host address space. Forgetting this is what let bug #14 (identical-VA collision between two guest processes) back into the C. But under the single store they stop being *memory owners* and become **address-space containers**: `Publish`, `PublishVidmem`, `JoinFbLeaf`, `AliasFbLeaf`, `FbJoinPeek` and `ExportBacking` all delete, leaving `MapStoreSlice`, `AdoptVaSpace`, `channel birth`, `doorbell` and `control`. □ **[UNVERIFIED]** Route K already births channels in the *scratchpad*’s process — if that holds for UVM’s channels too, VA identity has already moved and this conclusion changes.

2.2 The sandbox — two stages, forced order

Stage A, taken by the parent at clone [CODE spawn_unsafe.rs:1009]: CLONE_NEWUSER|NEWPID|NEWNET|NEWIPC|raw SYS_clone, then rootless uid/gid maps (setgroups=deny first, as the kernel requires). `execveat(AT_EMPTY_PATH)` on a **sealed memfd**, so the image cannot be substituted. `PR_SET_NO_NEW_PRIVS` before `exec`.

Stage B, taken by the child in its own main, before any worker thread exists [CODE sandbox_unsafe.rs:770-891] — and **the order is the security property**, which is why it is one call and not three: nested user ns → `MS_REC|MS_PRIVATE` / → 256 KiB tmpfs root (nosuid,noexec) → bind the two device nodes → `pivot_root(".", ".")` + `umount2(MNT_DETACH)` → **only now** open the `/dev dirfd` → remount root read-only → drop every capability and **read the result back**, a single surviving bit being a refusal.

□ **Three things any security claim must not overstate:** 1. **seccomp is NOT installed** [CODE lib.rs:57]. The fixed fd numbers exist *so that* a future filter can hardcode them; that filter does not exist. Any document saying the isolate is syscall-filtered is wrong. 2. **The CUDA scratchpad has a full filesystem view** for the duration of its CUDA bring-up, which runs *before* `sandbox::enter` [CODE child.rs:355-374]. Constraint 20 must be read as “*it ends with the same reach*”, not “*it never had more*”. 3. That bring-up makes the child **multi-threaded** before `sandbox::enter`, and `unshare(CLONE_NEWUSER)` is refused to a multi-threaded process — so the scratchpad depends on the fallback arm under inherited `CAP_SYS_ADMIN`.

□ [CODE isolate.rs:20-32] fd 4 is **deliberately vacant**. It used to carry an `O_PATH /dev dirfd`, and `openat(4, "../etc/shadow")` **opened** — `O_PATH` places no restriction on ...

2.3 The threads

Exactly two production threads are created by kayfabe inside the VMM [CODE]:

thread	loop	waits on
kayfabe-doorbell-publish	doorbell_publish_loop	Condvar on the publication queue
kayfabe-completion-observer	observer_loop	<code>epoll_wait</code> , 250 ms tick, woken early by an eventfd

Plus QEMU’s own main and vCPU threads, on which our trap code runs.

Inside each isolate child: a main thread, one **control** thread blocked in `recv(2)` on a datagram socket, and `--workers` (default **4**) pool threads each blocked reading its own socket. □ **None of the child’s threads are named**, so `/proc/<pid>/task/*/comm` shows `kayfabe-isolate` for all of them.

□ The control thread is separate for one reason: a cancel must be serviced **while every worker is busy**, which is the only state in which a cancel is ever wanted. It signals with `tgkill(SIGUSR1)` whose handler is installed with **no SA_RESTART** — “*the single most consequential line in the file*” [CODE signal_unsafe.rs:63] — because that is what makes an in-flight `RM ioctl` return `EINTR`.

□ **[UNVERIFIED]** `observer_loop` declares no `ThreadClass`, so it defaults to `Coordinator`, under which long blocking sections are a reported violation — yet it deliberately performs millisecond-scale budgeted drains. The publication worker *does* declare `Worker`. There is no comment either way.

3. The vCPU trap path, step by step

3.1 How a guest access reaches Rust

KVM `vmexit`

```

→ nvkvm_trap_write()          qemu/hw/misc/nvkvm/nvkvm.c:591    (MemoryRegionOps.write)
→ kayfabe_shim_regs_write()    shim_unsafe.rs:1371            (extern "C")
    mark_vcpu_thread()          :1384                        thread-local
    TrapGuard::enter_at(bar<<56|off) :1453                thread-local depth
→ Regs::write()                shim.rs:18576
→ RegPlane::write()             plane.rs:6473

```

Reads are shorter: nvkvm_trap_read → kayfabe_shim_regs_read → Regs::read → RegPlane::read.

∅ BAR0 is **cut into pieces**: 4 KiB pages whose every dword is “dead” get a RAM-backed alias so the guest’s reads of them never exit at all; the rest stay live trap pieces.

3.2 A DOORBELL write (BAR0 0x00BB0090) — the hot path

Defaults in force: doorbell_async = On (defers), doorbell_inline = Off.

1. RegPlane::write — two atomics (writes, mmio_in_flight AcqRel), then the doorbell arm matches and **returns before the rank-0 plane mutex is ever reached** [CODE plane.rs:6525].
2. ring_doorbell — bumps a counter; takes mmu_inval.inner briefly; takes plane.doorbell (**read**) across the call.
3. SharedDoorbell::ring — the passthrough inline short-circuit is off by default; shadow_route is pure arch math + one Relaxed load on an atomic slot array. **No lock**.
4. pubqueue.offer(MapPublication::for_doorbell(token)) — takes pubqueue.inner, coalesce check, cap check, VecDeque::push_back, **drops the guard**, then notify_one.
5. Back out: dbcost (atomics), **dbledger.seen** (a mutex, ≤64 tokens), and the report is Scheduled — so announce_completion is skipped and state/cpu_intr are **not** taken.
6. The tail of Regs::write runs on every write: a few Relaxed epoch swaps, each of which may emit one further offer.

Net on the vCPU: ~6 atomics, **4 short lock acquisitions**, one VecDeque push, one notify_one. No rank-0 plane lock, no device lock, no IPC, no blocking.

Deferred to the worker: everything — mirror revalidation, isolate spawn, ring adoption, the GP-FIFO read, the host forward or CE execution, the retired-object drain.

3.3 An MMU_INVALIDATE write (BAR0 0x00B830B0) — the one the guest spin-polls

1. RegPlane::write → the invalidate arm. Writes to the two PDB latches are pure latching.
2. On the **trigger**: note_trigger takes mmu_inval.inner, decodes, then pending.swap(true, AcqRel) — *this is the atomic the guest polls* — and bumps issued.
3. shadow_write pushes the new trigger value into the BAR0 backing page: takes plane.read_shadow (**read**) then ShadowSink::segments, then a write_volatile.
4. Regs::write offers MapPublication::for_invalidate onto the same queue.
5. **The vCPU returns. It never blocks.**

Where the guest blocks: in its own loop. ogkm spin-polls the trigger until it reads false, with a **4 s** (graphics) / **30 s** (compute) timeout. If the containing page is RAM-backed the poll costs **no vmexit at all**; if not, each poll is a read trap served by a deliberately lock-free arm. □ **[UNVERIFIED]** whether page 0xB83000 is backed for the shipped profile — the predicate is per-dword over the whole page and the survey did not execute it.

What clears TRIGGER: the worker, at the end of its pass — complete_through_unmaps(seq, ...), which **withholds** completion if a newer trigger arrived or if unmaps are outstanding (Constraint 27: completing early would let the guest reuse a page still reachable through a slice we have not torn down), then pending.swap(false) and a write_volatile of 0 into the shadow.

□ [CODE shim.rs:18726] the completion is **no longer a Drop guard**. In the code’s own words: “complete() is now the **ONLY** thing that clears the trigger, and a worker that dies without calling it hangs the guest by construction.”

3.4 Back-pressure — the rule that makes deferral safe

The queue never drops work silently. A refused offer converts a **notification** into a conservative **full sweep**: a dropped doorbell arms an emulated-channel sweep; a dropped invalidate arms a full rescan; a dropped RPC bind force-releases held replies. Owner’s rule, quoted in-source: “a lost NOTIFICATION becomes a conservative FULL RESCAN ... losing one may only ever cost time, never coverage.”

□ **Coalescing is OFF for doorbells, and that is a measured verdict.** [MEASURED] on real GA106 the coalescing arm produced cuCtxCreate → unknown error (999) and two host Xid 31s (CE2 FAULT_PDE, GR0_PBDMA0 FAULT_PTE); the nocoalesce arm was clean. “Deferring is fine and coalescing is not.”

4. Locks, and the blocking rules

4.1 The ranked order

[CODE lock.rs:56] “The total, one-way lock order ... Every lock declares its rank at construction; a thread may acquire only in strictly increasing rank, at most one lock per rank.”

rank	lock	protects
0	RegPlane::state	the GSP FSM, guest-RAM port, command policy
1	RegPlane::mem	the emulated framebuffer store + GMMU format
2	SharedDevice::state (RwLock)	the spine, routing maps, delivery pump
3	per-Proc mutex	one guest process’s channels/VA spaces
4	leaf (cpu_intr, executor inbox, recorder)	

Enforcement is real: check_acquire **panics before the OS acquire**, always on, not a debug_assert.

□ Why the plane sorts *below* the device despite being further out: the shipping order is plane→core and ranks must strictly increase, so the plane must sort first “or every vCPU MMIO trap in the device would panic.” The ABBA this fixed was **guest-buildable** — “A guest could build the deadlock by ringing a doorbell on one vCPU while touching a register on another.”

4.2 The blocking rules — what is actually enforced

rule	mechanism	teeth
R1 no blocking under a ranked lock	assert_lock_free	runtime panic, always on
R0 no blocking on a vCPU	assert_not_on_vcpu	□ census by default — panics only under KAYFABE_VCPU_BLOCK_FATAL
no publishing from a trap	OffVcpu token, taken by value	□ compile-time ; minted in exactly one place
INLINE-SAFE(site) predicate	prose only	∅ (a) and (b) have no mechanism anywhere

□ The `OffVcpu` doc is the clearest statement of the principle in the tree: *“A constraint that lives only in prose is re-litigated by every new path, and the reviewer who has to notice it is the weakest link in the system. **The type is the reviewer.**”*

□ **The read path does not mark the thread as a vCPU.** `kayfabe_shim_regs_read` installs only the `TrapGuard`; a thread that has never done an MMIO write is not `is_vcpu()`. A second, independent detector does catch it — *“the trap witness is measured, a declaration is a claim, and where they disagree the measurement wins.”*

4.3 ☐ The gate that is RED right now

`tests/tests/unranked_locks.rs` enumerates every **unranked** lock a vCPU can hold, because `assert_lock_free` masks only *ranked* locks and therefore **cannot see these at all**. Its own header is honest about what it buys: *“ENUMERATED, NOT ENFORCED ... Nothing fires when one is taken in a bad order.”*

[MEASURED 2026-09-20] it FAILS with **21 undeclared** unranked vCPU-path locks. Two matter most:

- `dbtable.rs` `Mutex<BTreeMap<u64, [u64; 4]>>` — **the doorbell ledger, taken on every doorbell.** ☐ Mine, added `w801`.
- `shim_unsafe.rs` `Mutex<Vec<ShadowSegment>>` — taken by the **vCPU** in `shadow_write` and by the **worker** in `publish_invalidate_trigger`. That is a lock genuinely shared between a trap and a worker, which is **precisely clause (c)** of the inline-safe predicate.

□ And if the BQL contradiction in §0 resolves toward *lockless*, two vCPUs can be in these paths simultaneously, which makes the shared-lock case sharper still.

[PROPOSE] Classify all 21 with real rulings — not to turn the gate green, but because the answer to *“can anything block beneath this?”* is a fact the v2 design needs anyway.

4.4 Where we block on purpose

[CODE `isolate.rs:454`], stated at the site: *“this is a synchronous write-then-blocking-read over a unix socket to the isolate child, and **every production caller is inside the vCPU’s MMIO trap** under the QEMU BQL = whatever the child spends here, **the guest pays.**”*

⇒ So the slogan “no blocking work in any MMIO trap” is **not** what the code does. There is one sanctioned blocking site and it is the isolate round trip. The store-map path explicitly refuses to be one of these, declining by name on a vCPU rather than stalling.

[MEASURED `w470`] the worst recorded stall was an isolate **spawn** on a vCPU — `Regs::write` → `materialize_pending` → `spawn_host` → `read_frame` → `__recv`, **1.62 s** inside an MMIO exit. That is why the scratchpad is brought up at realize, before the guest runs.

[PROPOSE] v2 should make this structural rather than sanctioned: no RM verb on a vCPU at all. Every such call becomes an offer onto the queue, and the guest waits on its own primitive — which §1 of the architecture says it already tolerates.

5. Data structures

5.1 The object graph — identity vs label

The whole file is built on one distinction, and it is the right one: **a guest handle is a label, not an identity**. RM recycles handle values with no quarantine and no free list, so the same 32-bit number names different objects over time.

type	what it is
<code>NodeKey { client, handle }</code>	a handle — per-client namespace slot. A label.

type	what it is
ResId(u64)	stable identity, minted at the origin alloc, never reused
ResourceKey { origin, incarnation }	the reportable identity; ordered origin-first so maps iterate deterministically
HandleRef::{Origin, Alias}	how a live handle came to reference its resource — recorded, never re-derived

```

struct Resource {
    node: RmNode, refs: BTreeSet<NodeKey>, pdb: Option<Pdb>,
    pdb_aperture: Option<Aperture>, gpu: Option<GpuId>, map_refs: usize,
    owner_kind: ClientKind, owner: ClientId, owner_incarnation: u32,
}

```

Liveness is refs non-empty **or** map_refs > 0. next_incarnation is “the smallest ordinal not held by a live resource at this handle value”, computed as a **range query**, not a scan — explicitly to deny a complexity DoS.

Ø RmGraph::apply **clones the whole graph** for rollback, because three faults are raised after mutation. The comment records it: ~24 % of an O(live objects)-per-event control plane. **[PROPOSE]** this is a named, deferred cost and v2 should fix it by validating before mutating.

5.2 The runtime model

```
Gpu { spine: Spine, system: Proc, procs: BTreeMap<ProcId, Proc> }
```

□ The concurrency contract is in the type, not a comment: a **per-proc** op takes the device *read* lock plus that proc’s mutex; a **spine** op takes the device *write* lock plus exclusive access to every proc, expressed as a ProcSet argument.

Proc is not a hardware concept. ProcId is “purely the label of one dup-connected component of declared user clients in the RM graph.” The stable name is ProcAnchor — the smallest client declaration in the component.

□□□ The distinction that earned its keep: clients (ClientKey) **labels** a component; client_ids (ClientId) **matches** one. Matching on handle values alone let a re-declared namespace **adopt the previous tenant’s proc** — its isolate, arena, host VAS and pending-release queue.

Vas carries the address plane; Channel carries the exec plane. Ø Channel::method_state is the one guest-driven state and is bounded **structurally** (a fixed array), not by a check.

5.3 The address table — what v2 deletes

```

pub struct AddressTable { map: IntervalMap<Row>, owner: Option<Pdb>, generation: u64, blockage }
pub struct Binding { kind: RegionKind, phys: u64, aperture: Aperture, host: Option<HostBacking> }

```

All four Binding fields are private with exactly two constructors — declared_by_guest (no host object) and real_gpu_memory (the only kind carrying one). Rows are **ranges**, not pages.

resolve(pdb, va) checks identity **first** (owns), then a predecessor lookup; a miss is a **fault**, never a heuristic. bind checks identity, then host-VA identity, then extent agreement, *before* anything enters the map.

[PROPOSE] This whole structure goes. Under v2 the walk produces a diff, the diff is mmap/munmap, and the **host page tables are the model**. Every field above is already in hand at the moment you act: the target is identity (store offset == FB offset), sysmem comes from a static layout, permissions come from the PTE just walked, and “it went away” is munmap.

□ The machinery is **already built and unwired**: kayfabe_mmu::walkdiff::diff exists fully implemented with **zero production callers** (its only caller is a #[test]), MapOp::{Map, Unmap, Remap}

exists, and `StoreMapPort::apply_ops` takes a **list** and does unmaps-before-maps — while its one production caller always passes a single-element `Map`. The source even says “*when the delta lands, this call site does not change.*”

∅ **Consequence today:** the only production unmap is a same-VA re-point. `release_vas` has zero callers. So `StoreMapPort::outstanding` grows **monotonically over the VM’s life** — mappings only ever accumulate, which is the other half of the cross-client leak.

5.4 Crate layering

`util` → `arch` → `abi` → `{mmu, core, fwd, rmrpc, device, rt}` → `qemu-raw`, with `vmm` and `isolate` as ports.

□ One layering fact worth arguing about: **kayfabe-mm**u depends on **kayfabe-isolate**, because `Binding` carries a `HostHandle`. So the address plane sits *above* the sandbox port rather than beside it. **[PROPOSE]** deleting the address table inverts that and makes `kayfabe-mm`u a leaf crate — which is a good structural signal that the deletion is the right shape.

5.5 The isolate boundary — two layers, not one

□ A correction to a premise I held: `VerbPlan/VerbReply` are the **in-process** typed plan and reply. The **IPC** is `Request/Reply/Envelope`.

- **Transport:** one `SOCK_STREAM` socketpair **per pool worker**, granted at fixed fd numbers.
- **Framing:** u32 LE length then body, in **one** `write_all` — because a length written separately from its body permanently desynchronises a signal-interrupted channel.
- **No multiplexing, no demux, no txn correlation.** Each worker owns its socket and the channel is 1-deep. An explicit divergence from the C, which multiplexed 16 workers over one fd.
- ∅ Two `RmError` variants deliberately have **no wire form**: `ForeignHandle` (our own gate produces it before a request is written — a compromised child must not be able to claim our invariant broke) and `Wedged` (a child that could send it would be reporting that it never answered).
- **Cancel** travels out-of-band on a separate datagram socket, guarded by a `txn` the worker publishes immediately before the verb and clears immediately after — without it a cancel races the completion and lands on an unrelated later operation.

6. Installing a mapping, and observing a completion

6.1 The mapping chain, end to end

The decision is made by a **publication pass**, not by a fault — we map ahead, we do not wait to be asked.

```
doorbell_publish_loop                                (worker thread; mints the one OffVcpu)
→ publish_vas_rows(token, seen, OffVcpu) shim.rs:13259  – takes OffVcpu BY VALUE
→ vas_publish_census(...)                          – produces candidate (va, len, phys)
→ join_one_fb_leaf(...)                             shim.rs:15099
→ plane.fb_join_plan(phys, len) → FbJoinPlan::DeviceBacked { at: phys }
→ map_store_slice_for_leaf(...)                     shim.rs:14765
  step 0 decline if on a vCPU or in a trap           :14787
  step 1 vaspace_handover → scratchpad dups the space (once per Vas)
  step 2 apply_ops(store_vas, [MapOp::Map{ va, gpga: at, len }])
→ StoreMapPort::map                                 storemap.rs:559
→ iso.with_worker(|w| w.with_rm(|rm| rm.map_store_slice(...)))
→ ProxyRmBackend::map_store_slice → write_frame / blocking read_frame (process boundary)
→ child: worker_loop → serve_one → HostRmBackend::map_store_slice
```

```
→ RmConnection::raw_map_dma_slice      rm.rs:4425
→ ioctl(NV_ESC_RM_MAP_MEMORY_DMA)      rm.rs:4509      ← THE SYSCALL
```

□ **The syscall happens in the scratchpad isolate child.** The VMM performs no RM ioctl for a store slice; it writes and reads one socket frame.

Two details that are load-bearing and easy to lose: - ∅ The code takes **RM’s answer, not its own request** (`done.placed.first()`). The comment records that substituting the requested address here regressed the guest to NEVER RETIRED. - ∅ **Constraint 28:** if RM places the mapping anywhere other than where we asked, the mapping is **torn down** and the call refuses — a fixed map that silently moved is worse than none. - ∅ `status == 0x51` on a FIXED map means *already mapped there* (occupancy, not capacity) and is re-reported as such.

Guest RAM takes a different route: the memfd arrives **at spawn on a fixed fd**, not on a request, so a future seccomp filter can hardcode it. The child mmaps it and wraps it in an `OS_DESCRIPTOR`; ∅ RM pins those pages and releasing the mmap does **not** unpin them.

6.2 The single store

[CODE] Allocated at **PCI realize**, in the scratchpad, by a **bisecting probe**: halve to a floor, then bisect to 64 MiB grain, each attempt allocated and freed. First try contiguous + 1 GiB aligned; fall back to non-contiguous + 4 KiB, latching which — and that latch is what later picks the page-size flag for every mapping.

[MEASURED] **11904 MiB**. □ Not a constant anywhere: the compiled advertisement is 12288 MiB and is **rebound to whatever the probe reserved**. □ The identity guest FB offset == store offset is only safe *because* of that rebinding.

∅ Deliberately **not** `alloc_vidmem`: that verb is contiguous-with-alignment = len, so an 11.8 GiB request refuses on **fragmentation** — a refusal that is not about capacity, on the one call whose refusal is supposed to mean “there is not enough video memory.”

6.3 Completions — two mechanisms with opposite writers

path	who writes the value the guest polls	how we learn
GR / compute , forwarded	the GPU	we only <i>read</i> : <code>epoll_wait</code> at a 250 ms floor, woken early by an eventfd; the watch list is declared on the vCPU and swept on the observer thread
emulated CE , served locally	we do	our executor writes the guest’s own literal payload, at the address the guest’s own page tables resolve to, after the bytes moved

□ The completion-watch module **never writes, never raises, never resolves** — the capability it is handed is a reader, by construction. □ The CE writer is a **single-writer discipline** with a compile-fail test and a workspace-wide census, because the C had a second, lagging writer that corrupted a live payload.

∅ `AWAKEN_ENABLE = 0` in the measured `cuCtxCreate` stream: the guest asked for **no interrupt at all**. The completion is a value appearing at an address. That is why polling — not interrupt delivery — is the mechanism here.

6.4 Interrupts — what gets announced

Two independent gates, both in ring_doorbell: - **non-stall vector**: only a **locally served** doorbell announces. A forwarded one does not — the work finished on a host engine at an instant this device was not standing at. - **OS-event wake**: fires for **served or served-locally**. [MEASURED w686] gating this on locally-served alone meant **40 of 44** completions on passthrough channels announced nothing and the guest sat in MC_SERVICE_INTERRUPTS.

Ø LEAF_TRIGGER is the **guest's own** self-trigger register, not our announcement path. Masking is **recorded, not acted on**.

7. The proposal, and the arguments against it

This is the part to attack. Each item has the counter-argument I can see.

P1. Mirror, not adopt

There are two ways to make the GPU honour the guest's translations.

- **Adopt** — point the host channel's instance block at the **guest's own** page-directory root. No walk, no diff, no syscalls at all. Most of v2 evaporates.
- **Mirror** — walk the guest's tables, replicate into a host VA space.

I propose mirror, and I want this one argued. My objection to adopt: the guest's PTEs contain **guest** physical addresses, which are host-valid only where the backing is identity — true for the store, false for system GPAs. □ If adopt can be rescued (e.g. by making system identity too), it is strictly better and I would drop mirror.

P2. No address table; the host page tables are the model

Already argued in §5.3. The strongest evidence it is right: **the diff machinery already exists and is unwired**, and the reconciliation bugs that dominated 2026-09-19/20 are all of the form “*two copies of one fact disagreed*.”

Ø **Argument against**: syscall volume. N changed pages $\Rightarrow$ N mappings unless runs are coalesced. **Unmeasured**. The walk is 205.7 μ s and page tables change far less often than doorbells ring, so I expect small bursty diffs — but that is a prediction.

P3. Lazy everywhere; strict only at recycle — and recycle is the guest's job

A doorbell **never** waits for a refresh. ogkm already accepts that addresses are *undefined* until an invalidate completes: old page or new page, never a page neither mapping referenced.

$\Rightarrow$ We own no scrub policy. Our single failure is that we **forge the scrub's completion** (forwarded=0 on the kernel tokens), so the guest reuses a page it believes was cleared. **That is the cross-client leak**. Fix: let the work reach the GPU. The gate already exists.

P4. No RM verb on a vCPU, structurally

§4.4 shows the current design **sanctions** one blocking site and pays for it with a measured 1.62 s stall. **[PROPOSE]** make it impossible instead: every RM verb becomes an offer, and the guest waits on its own primitive.

Ø **Argument against**: some verbs are genuinely synchronous in the guest's eyes — a control command whose reply the guest reads immediately. Those need an answer *now*. □ I do not have a clean answer for that case and it is the main hole in P4.

P5. Deletion order

1. **The scrub** — live security bug, gate exists, smallest change.
2. **Pdb-as-identity** — still costing diagnoses; split into `VasId + Option<Pdb>`.
3. **The address table and joins together** — they are one mechanism.
4. **The CPU executor last** — the scrub result tells us which half of that plane survives.

P6. What must NOT be deleted

The instruments. The doorbell ledger's per-token forwarded=, the censuses, refusals **by name**, the bare-metal baseline, the hardware gate. [MEASURED 2026-09-20] the bare-metal baseline is what turned nine guest failures from opinion into adjudication; without it, a rewrite cannot be graded, and a rewrite you cannot grade is a rewrite you cannot land.

8. Open problems — named, not designed away

1. **Where the walk's root comes from.** `MAP_MEMORY_DMA` is a HAL stub on GSP parts, so a client-allocated VA space is never declared — [MEASURED] **zero** `SET_PAGE_DIRECTORY` events in a whole boot. Candidates, ranked: the **instance block** (which we author, and which is how hardware itself finds the root); `COPY_SERVER_RESERVED_PDES` (arrives, but only for RM-managed spaces); observing the guest's own page-table writes.
2. **Fault delivery becomes load-bearing.** v2 lets the GPU fault instead of pre-validating operands — but guest fault-recovery methods are currently **refused by name** precisely because we do not deliver faults. That refusal is the marker for this work.
3. **The BQL question** (§0). It changes the concurrency argument wholesale.
4. **The 21 unranked vCPU-path locks** (§4.3).
5. **Multi-GPU and non-GA10x.** The chip table holds exactly one profile.
6. **Syscall cost of the diff** (P2).

Part 3 — The surface we present

STATUS: LIVE, 2026-09-20 (w815). Companion to THE_ARCHITECTURE_v2.md.

∅ Every row below was read out of the tree at w749-fable-legb, with file:line. Where the survey could not establish something it says so — **an unverified row is marked □, never filled in**. This document is the inventory the v2 rewrite is scoped against: the **plan** column is the decision, and [DELETE] means the mechanism goes away, not that the behaviour does.

1. GSP RPCs

The wire table is generated from `rpc_global_enums.h` (kayfabe-abi/src/generated/rpc.rs); classification is `FunctionCodes::classify` (kayfabe-gsp/src/rpc.rs:209); the one dispatch is `translate()` (kayfabe-rmipc/src/lib.rs:1073).

1.1 The object-model RPCs — KEEP, unchanged

GSP_RM_ALLOC (0x67), GSP_RM_CONTROL (0x4c), FREE (0x0a), DUP_OBJECT (0x15).

These four are the whole guest→us object protocol: the guest tells us what it allocated, what it freed, what it aliased, and what it wants done. We decode them into `RmEvents` and keep a graph of the guest's objects. **Plan: unchanged in v2.** This is the control plane, it is version-sensitive, it is heavily tested, and none of v2's deletions touch it. An unmapped alloc class is refused by name (`UnmappedAllocClass`) and that stays — a class we do not model is a thing we must not pretend to have created.

1.2 UPDATE_BAR_PDE (0x46) — KEEP, but it is the BAR2 root

Inert to the object model; acted on at `kayfabe-device/src/bar2.rs:299`, where it latches the root PDE that BAR2 translations are rooted at, and replies `NV_OK` with an empty body. **Plan: keep.** BAR2 is an instance-block window and the guest legitimately re-roots it. □ This is one of the few places v2 still walks page tables *to answer a read*, rather than to produce a diff — see §3.4.

1.3 SET_GUEST_SYSTEM_INFO (0x01) / _EXT (0x40) — KEEP

The version handshake. We answer with the version we agreed to speak; the `_EXT` form is answered `NV_OK` with an empty body. The guest's firmware string is latched read-only. **Plan: keep** — this is where the support matrix is enforced, and it must stay strict.

1.4 GET_GSP_STATIC_INFO (0x41) — KEEP, forged by construction

We synthesise an entire `GspStaticConfigInfo` from the chip and driver tables (`staticinfo.rs:183`). There is no host value to forward: on real hardware this comes from GSP firmware we are replacing. **Plan: keep forged.** □ This is the single largest fabricated structure we hand the guest and it is derived per-die; it is the first thing to re-derive when adding a chip.

1.5 The inert set — KEEP as inert, they are load-bearing

INIT_GSP_TRACE_CRASH_BUFFER (0xe4), UNLOADING_GUEST_DRIVER (0x2f), GSP_SET_SYSTEM_INFO (0x48), SET_REGISTRY (0x49), ECC_NOTIFIER_WRITE_ACK (0xca).

Accepted, no state moves. ∅ Two of these matter more than “inert” suggests: `UNLOADING_GUEST_DRIVER` **requires a reply** — a missing one hangs `rmmod` — and the two `NoReply` ones must **not** be answered, because an unsolicited reply desynchronises the queue. **Plan: keep, with the reply disposition as the thing under test.**

1.6 Events we post — KEEP

EVENT_GSP_INIT_DONE (0x1001), EVENT_POST_EVENT (0x1003), EVENT_RC_TRIGGERED (0x1004). We send these; a guest-sent one is refused EventFromGuest. **Plan: keep.** The refusal direction is a real boundary — the guest must not be able to inject its own completions.

1.7 Everything else — refused by name, and that is the design

Other(code) ⇒ UnknownFunction { code } — *replied to, never dropped*. Includes ids present in the generated table with no dispatch arm: NOP, ALLOC_ROOT, ALLOC_MEMORY, MAP_MEMORY_DMA (0x0e), UNMAP_MEMORY_DMA (0x0f), EVENT_GSP_RUN_CPU_SEQUENCER.

□□□ MAP_MEMORY_DMA is the important one. It is a HAL stub on GSP-client parts, so the guest never sends it — which is why [measured w811] a whole boot carries **zero** SET_PAGE_DIRECTORY events and a client-allocated VA space is never declared to us. **Plan: this is open problem #3 in THE_ARCHITECTURE_v2.md §7.** v2 needs a root to seed a walk and this RPC will not provide it.

2. RM control commands

2.1 Page-directory controls — [DELETE the concept, keep the decode]

cmd	name	today
0x00801813	DMA_SET_PAGE_DIRECTORY	→ RmEvent::SetPageDir
0x90f10106	VASPACE_COPY_SERVER_RESERVED_PDES	→ RmEvent::SetPageDir
0x20800a9f	GMMU_COPY...PDES_TO_SERVER	same params as above
0x00801814	DMA_UNSET_PAGE_DIRECTORY	refused
		PageDirControlNotModelled

These tell us a VA space’s page-directory base. **Plan: keep decoding them, but demote what the answer is used for.** In v2 a root is needed **only to seed a walk** — never as an address space’s identity. **[ROT]** today Pdb is both, and unwrap_or(Pdb(0)) at eight sites makes Pdb(0) mean “*declared at offset 0*” and “*not declared*”; that collision cost four wrong diagnoses in one session. □ Note both arms refuse a **sysmem** root today (SetPageDirRootAperture / PublishedPdesRootAperture) even though a real GA106 was measured with one — that refusal is on the v2 review list.

2.2 GPU_PROMOTE_CTX (0x2080012b) — KEEP

The guest tells us where it placed a context buffer. Decoded, ≤16 entries, entryCount > 16 refused. **Plan: keep.** □ The C rounded every promote-derived mapping **up to 64 KiB** (asize = (size + 0xffff) & ~0xffffull); this port binds at the declared length and that difference produced sub-page holes. Under v2 the mapping comes from the **walk**, not from the promote, so this becomes an observation rather than a mapping source — which dissolves the rounding question instead of answering it.

2.3 Channel controls served by ObjectPolicy — KEEP

GPFIPO_SCHEDULE (0xa06f0103 / 0xa06c0101), BIND (0xa06f0104), SET_CTXSW_PREEMPTION_MODE (0x20801210), MC_SERVICE_INTERRUPTS (0x20801702), GET_MMU_FAULT_INFO (0x906f0106), PREEMPT (0xa06c0105).

The channel lifecycle: schedule it, bind it to an engine, preempt it, ask why it faulted. **Plan: keep all.** ∅ GET_MMU_FAULT_INFO is **relayed to the host channel verbatim, never synthesised**, and that becomes *more* important in v2: if the GPU faults because our diff was late, the guest must receive the hardware’s own answer.

2.4 The 47 forged tables — KEEP forged, but they are a per-die liability

FIFO_GET_DEVICE_INFO_TABLE, INTERNAL_GPU_GET_CHIP_INFO, CE_GET_CE_PCE_MASK, FB_GET_INFO_V2, GR static caps, the cudart-init group, and ~40 more (kayfabe-device/src/inittables.rs:1085). Each is answered with a struct built from our chip and driver tables; a paramsSize mismatch is refused.

Plan: keep, and derive per die rather than maintain per die. □ This is the largest correctness surface we own and the one with the worst oracle: [the C artifact] 11 of 56 captured rows had **dlen = 0** and every empty row checked against real hardware was **contradicted**. CE_GET_FAULT_METHOD_BUFFER_SIZE decoded from an empty row as **0** where a real GA106 returns **20480** — and RM DMAs fault records into a buffer of that size, so trusting it was a buffer overrun with a hardware writer.

2.5 Input-only controls — KEEP

NV2081_BINAPI, DEBUG_SET_EXCEPTION_MASK, SET_TIMESLICE, two CUDA-limit controls. The ack is the whole verb; we echo the guest’s own bytes behind a size check. **Plan: keep.**

2.6 Denied by capability — KEEP DENIED

IMPORT_MEM, DISABLE_IMPORTERS, NVLINK_GET_PLATFORM_INFO (fabric management); GPU_EXEC_REG_OPS, NVB0CC_EXEC_REG_OPS (raw register access); REPORT_NON_REPLAYABLE_FAULT; the SM-debugger trapping set; ALLOC_PMA_STREAM (performance counters).

Plan: keep denied, permanently. These are the guest asking for a capability the host process does not have and must not acquire. □ This is a *capability allowlist* (160 rows), not a served set: admitted-but-undecoded still refuses GspRuleControlUnserviced. □ The survey did not transcribe all 160 rows — that is a mechanical dump if the full admitted set is ever needed.

3. MMIO registers

□ **GA106 only.** The chip table holds exactly one profile; ad10x, gh100, gb20x exist in kayfabe-chips and are not in it. □ The BAR0 aperture is 16 MiB and ~87% of its pages hold no register at all — “everything else ⇒ Unclaimed” is the complete statement for the remainder.

3.1 Boot constants — KEEP

NV_PMC_BOOT_0/1/42, NV_PTIMER_TIME_PRIV_LEVEL_MASK, NV_USABLE_FB_SIZE_IN_MB, NV_XVE_LINK_CAPABILITIES Constants the driver reads to identify the part. **Plan: keep, per-die.** ∅ One is an advertised fiction — the access-counter notify buffer size carries an entry count only.

3.2 The GSP boot FSM — KEEP

GFW_BOOT_PROGRESS/PLM, the PGSP/PSEC falcon registers (CPUCTL, HWCFG2, DMATRFCMD, MAILBOX0/1, IRQSTAT/MASK/DEST/SCLR), NV_PRISCV_*, WPR2_ADDR_LO/HI.

This is the fake GSP’s entire visible boot: the driver pokes STARTCPU, watches progress reach 0xFF, reads mailboxes for boot args, and finds WPR2 where it expects. **Plan: keep.** ∅ WPR2 state only resets on a full QEMU restart, which is why each clean run needs a fresh boot.

3.3 Queue doorbells and the interrupt tree — KEEP

NV_PGSP_QUEUE_HEAD(i) (0x110C00 + i*8) is the command doorbell: posted to a worker, no state touched inline. CPU_INTR_LEAF(i) / EN_SET / EN_CLEAR, CPU_INTR_TOP*, and CPU_INTR_LEAF_TRIGGER (write-only, latches a vector and raises the CPU interrupt).

Plan: keep. ∅ [measured w684] 40 of 44 completions were never announced (no_engine); the interrupt **arming** model is the live design question here, not the register decode.

3.4 The MMU invalidate trio — [DELETE most of what it triggers]

MMU_INVALIDATE_PDB (0xB830A0) and _UPPER_PDB latch; MMU_INVALIDATE (0xB830B0) **triggers** a publication, and the guest **spin-polls** its TRIGGER bit for completion.

Plan: keep the register, delete the publication behind it. In v2 an invalidate is a hint that the guest's tables changed — a good moment to walk and diff — and **nothing waits on it** (§1: the guest already tolerates undefined-until-complete). ∅ Today this register is the entry point to the epoch/dirty-gate/publication machinery that §3 of the architecture deletes. □ RM cannot express a narrow invalidate (ALL_VA is hard-coded) and its own timeout is **4 s**, after which it proceeds with stale TLBs — so the guest is *already* designed for us to be slow here.

3.5 The framebuffer windows — [DELETE]

NV_PBUS_BAR0_WINDOW (0x1700) plus the PRAMIN moving window (0x700000–0x7FFFFFF); BAR1 (bus_bar::FB) and BAR2 (bus_bar::INST), both GMMU-translated on access; FbTrapPolicy.

Plan: BAR1/BAR2 become device views of the one store and stop being translated windows; PRAMIN stays as the bring-up aperture it is, not a running path. ∅ [measured] vidmem CPU reads are **~48 MiB/s flat** — any design that routes real traffic through these is slow by construction, which is precisely why the store exists.

3.6 The doorbell — KEEP, it is the hot path

NV_VIRTUAL_FUNCTION_DOORBELL (BAR0 0x00BB0090). Rung off-lock; drives channel submission. NV_VIRTUAL_FUNCTION_TIME_0/1 beside it are the PTIMER: read-only, **writes refused by name** (the guest reads the host's counter and may not move it). **Plan: keep all three exactly.** ∅ §41 bounds what this trap may do: update a queue, wake a worker, one dword for a passthrough doorbell, or a synchronous read-register write. Nothing else, ever.

4. Channel / pushbuffer operations

Decoder: Ga10xPushbuffer (kayfabe-chips/src/ga10x.rs:1595); vocabulary PushMethod (kayfabe-arch/src/lib.rs:949); consumer apply_pushbuffer (kayfabe-fwd/src/lib.rs:8721).

4.1 EMULATED — we implement the effect

method	addr	why it is ours
SET_OBJECT	0x0000	pure bookkeeping — binds a subchannel class, clears its slots
host-FIFO semaphore run (SEM_ADDR_LO/HI, SEM_PAYLOAD_LO/HI, SEM_EXECUTE)	0x5C–0x6C	RELEASE only ; acquires and reduction decode Opaque
MEM_OP_A..D TLB invalidate	0x28...	recorded and censused; PDB_ALL and non-invalidate ops ⇒ Opaque
LAUNCH_DMA with DATA_TRANSFER_TYPE_NONE	0x0300	a release carrying no copy — there is no transfer to forward

Plan: keep all four emulated. Each is either bookkeeping or a completion, never a byte movement. ∅ In v2 the TLB-invalidate record becomes a *walk trigger* and loses its publication side effect (§3.4).

4.2 TRANSLATED — operands rewritten, the GPU moves the bytes

LAUNCH_DMA (0x0300) is the only true member, fed by fourteen latched operand slots: OFFSET_IN/OUT_UPPER/LOWER, LINE_LENGTH_IN, LINE_COUNT, SET_SRC/DST_PHYS_MODE, SET_REMAP_CONST_A/B, SET_REMAP_COMPONENTS, SET_SEMAPHORE_A/B/PAYLOAD/_UPPER.

Today we re-resolve src/dst through the issuing channel's **address table** and partition into spans. **Plan: [DELETE the translation].** Under v2 the guest's VA is the host's VA (route K) and the store is identity-mapped, so there is nothing to rewrite — the GPU resolves the operands through the page tables we applied, and faults if we were wrong. ∅ This deletes TableOperands, the span partitioning, the operand gate, and CeExecutor::Ours with it.

□ SET_SRC/DST_PHYS_MODE is the one that must **not** be waved through: PHYSICAL bypasses the MMU, and a physical-mode copy passed straight to hardware is an escape class. It stays inspected — that is exactly what a **Translated** channel is for, and why Passthrough is not allowed to carry it.

∅ SET_REMAP_COMPONENTS is **required, never defaulted** — an unlatched remap refuses the launch. Keep that; a defaulted remap silently writes the wrong bytes.

4.3 REFUSED — the fault tripwire

GP100_UVM_SW_FAULT_CANCEL_A/B/C (0x0104/0108/010C) and CLEAR_FAULTED_A/B (0x0110/0114) abort the whole parse with UvmFaultMethodWithoutFaultDelivery. SET_OBJECT/NO_OPERATION on the same class pass through as routine.

Plan: keep refusing — and revisit when v2 lands. These say the guest is doing fault *recovery*, which presumes a fault-delivery mechanism we do not implement. □ v2 deliberately lets the GPU fault (§3), so fault delivery moves from “not modelled” to “on the critical path”, and this refusal is the marker for that work.

4.4 PASSED THROUGH

Everything else ⇒ PushMethod::Opaque, counted, untouched. Non-Incrementing method forms (NON_INC, ONE_INC, IMMD_DATA) are deliberately not latched. **Plan: keep.** ∅ Parsing — not placement — is what is forbidden on a passthrough ring; this is the rule in code.

5. □ Where this inventory is knowingly incomplete

1. The 160-row capability allowlist is not transcribed (gates admission, not service).
2. Registers are **GA106 only**; three other chip modules exist and were not enumerated.
3. There is no enumerable table of *unhandled* BAR0 offsets — ~87% of pages are Unclaimed.
4. kayfabe-rt/src/ceutils.rs holds a **second** PushMethod consumer; it was confirmed to use the same enum but **not** audited for additional method addresses.
5. kayfabe-isolate-host/src/rm.rs *emits* CE pushbuffers using the same constants; classified as a producer, not audited as a decoder.
6. Roughly a dozen kayfabe-device modules were not read in depth; two of them gate on RpcFunction::RmControl and may reference control ids not listed here.

Part 4 — The scrub question (open owner ruling)

STATUS: OPEN — awaiting an owner ruling on §46 vs §12.26. Measured 2026-09-20 (w813) on a GA106 / 580.159.04 guest, branch w749-fable-legb. Nothing here has been implemented; the SystemDataPlane rule is untouched.

What is measured

The DOORBELL-LEDGER, per token, on a thin-guest run whose every ledger row verified:

token	class	emulated	forwarded	reading (the ledger's own rule)
0x00000003– 0x00000007 0x00000009	the guest's CE channels guest	1 2	5 2	reached hardware reached hardware
0x00010001	kernel / system	74	0	never reached hardware
0x00010004	kernel / system	66	0	never reached hardware

“forwarded=0 with emulated>0 means it never went to hardware; forwarded>0 means it did” — the ledger's own text.

⇒ **The guest's copy work runs on the GPU. The kernel's does not, and it is 140 doorbells.**

Why this is a decision and not a bug

The kernel tokens are forwarded=0 **by design today**. kayfabe_fwd::FwdFault::SystemDataPlane implements l1_concurrency.md §12.26:

*“The SYSTEM proc has no data plane ... Guest-kernel work that would need a backing — **the CeUtils scrub**, the GR golden capture — is **forged** to the system proc's completion queue, **never forwarded**, so the system proc never mints host memory.”*

□ §46 (owner, 2026-09-19) names that **exact** example and reverses it:

“okay no real work kernel channel except the no-op or the ones that we must stub, so the scrub and CE, is never executed on CPU. Its always executed on GPU under the single store” and, when I mis-paraphrased it: “Ceutils scrub is not a no-op, it genuinely needs to clear memory. Same for kernel ce”.

Two rules, one workload, opposite answers. The date is part of the citation, and §46 is the later one — but §12.26 is documented as a boundary, so the supersession should be **stated** rather than inferred by whoever next touches the code.

The argument each way

For §46 superseding (my reading): - §12.26's stated *reason* is “so the system proc never mints host memory”. Under the single store that premise is obsolete: the store is **one reserved device-local object minted once at realize**. A guest-RAM pin against it mints nothing; it names an offset in something that already exists. - The scrub is real work with an observable end state — memory that must read as cleared. §46 is explicit that this is not a no-op we may fake. - The same reasoning already moved the guest's CE copies onto hardware, and nothing about the system proc makes its bytes different bytes.

Against / what to be careful of: - §12.26 is about a **lifetime regime**, not a byte count: the system proc's isolate exists for the device's own bring-up and is not torn down with any guest process. Handing it data-plane state is a new lifetime question — *what frees these pins, and when?* - ce_executor_tree.md (owner, 2026-08-07) separately rules that ce_copy(0urs) must keep refusing inside the isolate, calling it *“the security boundary refusing to leak guest memory into the sandbox, working as designed”*. ∅ That is a **different** refusal from SystemDataPlane and is **not** what this page proposes changing — but the two are adjacent and must not be conflated in a hurry.

If the ruling is “§46 wins”, the work and its gate

1. plan_pin_guest_ram stops refusing Gpu::SYSTEM_PROC — and the lifetime question above gets an answer in the same change, not later.
2. The scrub's CE doorbell forwards like any other.
3. **The gate already exists and needs no new instrument:** tokens 0x00010001 and 0x00010004 must flip from forwarded=0 to forwarded>0 in the DOORBELL-LEDGER, and run_fast_guest.sh's stranded-token check must be widened from guest tokens to all tokens. □ Until then that check is deliberately scoped to tok=0x000000xx **precisely so it cannot pre-empt this ruling** — a gate that asserted the answer would make the question unaskable.

What must not be read into this page

∅ This is not a claim that the scrub is broken. It is forged, deliberately, by a rule that was correct when written. The question is only whether the rule survives the single store.